

C Programming Mini Project Report

Cinema Seat Reservation System

Introduction

Cinema halls handle a large number of seat bookings every day, and doing this manually makes it difficult to track seat availability in real time, avoid double-booking, and calculate revenue accurately. This project presents a console-based Cinema Seat Reservation System developed in C, which automates the process of viewing seat availability, booking and cancelling seats, searching bookings by customer name, and generating revenue and occupancy summaries.

The system is designed as a menu-driven application with two access levels: a User menu for customer-facing booking operations, and an Admin menu for managing the list of movies and viewing summaries. The project was built to apply and demonstrate core C programming concepts as well as fundamental data structures and algorithms, including arrays, linked lists, hashing, sorting, and searching, within a practical, real-world scenario.

Problem Statement

Section	Description
Existing Problem	Cinema ticket counters often allocate seats manually or using printed charts, making it difficult to track real-time seat availability
Proposed Solution	Develop a console-based Cinema Seat Reservation System using C Programming to automate seat booking, cancellation, searching, and revenue tracking.
Target Users	Cinema Booking Counter Staff
Expected Benefits	Faster seat booking, elimination of double-booking, quick lookup of customer bookings, and instant revenue/occupancy summaries.

Requirement Analysis

Project Objectives

Objective ID	Objective
OBJ-01	Display the current seat availability map
OBJ-02	Book a seat for a customer
OBJ-03	Cancel an existing booking
OBJ-04	Search bookings by customer name
OBJ-05	Generate a booking summary (seats booked/available, revenue)
OBJ-06	Validate seat input and prevent invalid or duplicate bookings
OBJ-07	Apply differential pricing for Premium and Regular rows

User Stories

ID	User Story	Acceptance Criteria
US-01	As a staff member, I want to view the seat map so I can see which seats are booked and which are free.	Booked seats are shown blank; free seats display their seat code (e.g., A1).
US-02	As a staff member, I want to book a seat for a customer so the booking is recorded.	Seat is marked booked and the customer name is stored; already-booked seats are rejected.
US-03	As a staff member, I want to cancel a booking so the seat becomes available again.	Seat is freed and the customer name is cleared.
US-04	As a staff member, I want to search bookings by customer name so I can quickly find a customer's seat.	All seats booked under the given name are displayed; a clear message is shown if none are found.

ID	User Story	Acceptance Criteria
US-05	As a manager, I want to view a summary of bookings so I can track occupancy and revenue.	Total seats, booked seats, available seats, and total revenue are calculated correctly.

Functional Requirements

Requirement ID	Requirement	Status
FR-01	View Seat Availability	✓
FR-02	Book a Seat	✓
FR-03	Cancel a Booking	✓
FR-04	Search Booking by Customer Name	✓
FR-05	View Booking Summary (Seats & Revenue)	✓
FR-06	Seat Input Validation	✓
FR-07	Differential Pricing (Premium / Regular)	✓

Project Modules

Module	Description	C Concepts Used
Seat Map Display	Show the current booking status of all seats	Arrays, Loops, Functions
Seat Booking	Book a seat and record the customer name	Arrays, Strings, Functions
Seat Cancellation	Cancel an existing booking	Arrays, Functions
Booking Search	Search bookings by customer name	Loops, String Comparison
Booking Summary	Calculate seats booked/available and revenue	Loops, Arithmetic
Menu System	Handle user navigation between options	Switch Case, Loops

Data Structures Used

The system combines several data structures to manage movies, seats, bookings, and customer loyalty records.

Data Structure	Purpose
Movie struct (title, booked[5][6], customerName[5][6][30])	Each movie stores its title, a 2D array marking which of the 5x6 seats are booked, and a parallel 3D array holding the name of the customer occupying each booked seat.
Dynamic array of Movie (movies, with movieCount / movieCapacity)	Movies are stored in a resizable array, doubled in capacity with realloc() as new movies are added, avoiding a fixed upper limit on the number of movies.
SeatNode linked list	Used as a temporary 'cart' while a user is booking multiple seats in one session; each node holds a chosen row/column and the list is walked to build the final booking before it is committed to the seat arrays.
CustomerNode hash table (customerTable[31])	An array of 31 linked-list buckets implementing separate chaining, used to track how many premium (front-row) seats each customer has booked, which drives the loyalty tier system.

Algorithm Design

Algorithm	Where Used	Description
Merge Sort	Keeping the movie list ordered (mergeSort, merge)	Whenever a movie is added, the movie array is re-sorted alphabetically (case-insensitive) using recursive merge sort, so the list stays in a predictable order and can support binary search.
Binary Search	Looking up a movie by title (binarySearchMovie)	Because the movie array is kept sorted, a new movie's title can be checked for duplicates in logarithmic time instead of scanning every existing movie.
Additive Hash Function	Customer loyalty lookup (hashFunction, loyalty)	A customer's name is mapped to a bucket index (0-30) by summing the ASCII values of its characters and taking the result modulo 31; collisions are resolved with chaining.

Algorithm	Where Used	Description
Seat Booking Algorithm	bookSeat	Reads seat choices one at a time, validates each against the seat grid and existing bookings, temporarily stores valid picks in a linked-list cart, shows a confirmation with the total price, and only then commits every seat in the cart to the booking arrays.
Input Validation Loop	Menu and seat input handling	Every scanf() call for menu choices and seat coordinates is checked for a valid read and range, with invalid input rejected and the input buffer cleared to prevent an infinite loop on bad input.

Implementation

C Programming Concepts Used

Concept	Implementation
Variables	Store row/seat indices, prices, and menu choices
Data Types	int, char
Arrays	2D array for booking status; 3D array for customer names per seat
Functions	Modular design (displaySeatMap, bookSeat, cancelSeat, searchBooking, showSummary, etc.)
Loops	Menu-driven operation and iteration over the seat grid
Switch Case	Menu selection
Conditional Statements	Seat validation and booking status checks
String Handling	Customer name storage and comparison (string.h, ctype.h)
Structures	Movie, SeatNode, and CustomerNode structs model movies, cart entries, and loyalty records
Dynamic Memory Allocation	realloc() for the growable movie array; malloc()/free() for cart and hash table nodes

Implemented Program Features

Feature	Description
View Seat Availability	Display the seat map with booked/free status and pricing per row
Book a Seat	Reserve one or more seats for a customer by row letter and seat number, with a confirmation step before the booking is finalised
Cancel a Booking	Free up a previously booked seat
Search Booking	Find all seats booked under a given customer name
View Booking Summary	Show total seats, seats booked, seats available, and revenue, for one movie or all movies
Loyalty Status	Track premium seat bookings per customer and report a Bronze/Silver/Gold/Platinum tier
Admin Movie Management	Add or remove movies, with the movie list kept sorted and duplicate titles rejected
Exit	Close the application safely, freeing all dynamically allocated memory

Menu System

The application is menu-driven and organised into three levels. The main menu lets a person log in as a User or an Admin, or exit the program. The User menu (unlocked after entering a valid alphabetic name) provides seat map viewing, booking, cancellation, booking search, and loyalty status. The Admin menu (protected by a username/password check) provides movie listing, addition, removal, and both per-movie and all-movie summaries. Each menu loops until the user selects the 'back' or 'exit' option, and every numeric menu choice is validated before use.

Complexity Analysis

Operation	Time Complexity	Space Complexity	Notes
Merge Sort (mergeSort/merge)	$O(n \log n)$	$O(n)$	n = number of movies; run each time a movie is added so the list stays sorted
Binary Search (binarySearchMovie)	$O(\log n)$	$O(1)$	n = number of movies; requires the array to remain sorted
Hash Function (hashFunction)	$O(k)$	$O(1)$	k = length of the customer name
Loyalty Lookup/Insert (loyalty)	$O(1)$ average, $O(m)$ worst case	$O(1)$ per node	m = number of names hashing to the same bucket (chain length)
Seat Map Display / Summary	$O(\text{rows} \times \text{cols})$	$O(1)$	Fixed at $5 \times 6 = 30$ seats per movie
Book / Cancel a Single Seat	$O(1)$	$O(1)$	Direct array index access after validation
Search Booking by Name	$O(\text{rows} \times \text{cols})$	$O(1)$	Scans every seat of the selected movie

GitHub Repository

Repository link: <https://github.com/samiiiiksha/Cinema-Hall-Seat-Reservation-System>

Results & Discussion

All functional requirements identified during requirement analysis were successfully implemented. The system correctly displays a live seat map distinguishing booked and free seats, allows booking of multiple seats in a single session with a confirmation step, supports cancellation, and allows customers' bookings to be searched by name.

Revenue and occupancy summaries are generated correctly for individual movies as well as across all movies, reflecting the differential pricing applied to the premium (front) row versus the regular rows. Input validation prevents invalid seat coordinates, out-of-range menu choices, and double-booking of an already-reserved seat.

An additional loyalty tracking feature, built on top of the hash table, rewards customers who repeatedly book premium seats with Bronze, Silver, Gold, or Platinum status, which was not part of the original functional requirements but adds practical value for a real cinema's repeat-customer program.

Keeping the movie list sorted with merge sort and looking it up with binary search keeps duplicate-title checks and movie management efficient even as the number of movies grows, rather than relying on a linear scan.

Conclusion & Future Scope

Conclusion

The Cinema Seat Reservation System successfully replaces a manual, paper-based seat allocation process with a structured, menu-driven console application. By combining arrays, linked lists, hashing, sorting, and searching, the project demonstrates how core data structures and algorithms can be applied to solve a practical, real-world scheduling and record-keeping problem while meeting all the functional requirements identified at the start of the project.

Future Scope

- Persist movies, bookings, and loyalty records to a file or database so data survives program restarts.
- Support multiple screens/showtimes per movie instead of a single fixed seat grid.
- Add a graphical (GUI) or web-based front end in place of the console interface.

- Introduce discount codes, group bookings, and seat-hold/expiry timers.
- Encrypt or hash the admin password rather than storing it as plain text.